GovMut-Health: A Mutation Benchmark for Sequence-Sensitive Governance Faults in Stateful Health-AI Systems

Nguyen Ngoc Thien and Trinh Minh Quang

Hai Ba Trung High School, Hue, Vietnam
kakangocthien109@gmail.com

Abstract. Stateful health-AI governance failures often depend on an ordered sequence of operations rather than an isolated malformed input. A system may behave correctly when a disclosure snapshot is compiled, consent is active, and a proposal is generated, yet still admit an unauthorized persistent write if patient consent is revoked, policy epochs advance, a stale proposal is retried, or un-invalidated cache rows are re-inflated before durability. While general mutation testing evaluates syntactic AST mutations (e.g., SWE-Mutation) and formal agent verification tests idealized state machines (e.g., MemTX), neither captures sequence-sensitive governance failures in stateful health architectures. GovMut-Health establishes the first domain-specific mutation benchmark for healthcare AI, mapping 15 multi-action defect classes directly to clinical data lifecycle invariants and cryptographic Merkle state transition verification. We formalize Theorem 3 (Cryptographic Merkle Digest State Transition Verification and Bound-Commit Security), proving that under standard collision resistance, proposals bound to stale or drifted states are rejected with overwhelming probability ($\leq 2^{-128}$). In a large-scale evaluation spanning a core curated corpus ($N = 45$) and a systematic domain expansion ($N = 1{,}440$ non-equivalent mutants across 12 clinical subsystems $\times$ 8 perturbations, totaling 23,040 test executions), mutation scores were 35.6% for regression tests alone, 8.9% for stateless properties, 13.3% for state-machine testing, and 100.0% for the combined GLHS governance strategy ($p < 10^{-50}$ vs. regression), demonstrating comprehensive fault-detection efficacy across complex multi-step clinical workflows.

1 Introduction

A stateful governance contract in healthcare AI can fail even when every individual API request satisfies local syntactic and schema preconditions. The defect typically requires an interleaved execution sequence: disclose patient context, modify consent or governing policy, retain a stale proposal, retry a previously conflicted write, and attempt durability. Standard unit and example-based regression suites frequently cover happy paths and known historical bugs, but they fail to systematically explore the combinatorial transition orderings where sequence-dependent governance defects emerge.

General mutation testing evaluates test suite quality by injecting syntactic faults into source code (e.g., SWE-Mutation [11]), while transactional agent memory benchmarks test idealized key-value state machines (e.g., MemTX [7]). However, neither paradigm models the domain-specific invariants of longitudinal health data: bitemporal valid-time and transaction-time intervals, clinical terminology ontologies (LOINC, SNOMED CT, RxNorm), dynamic patient consent revocations, and cryptographic Merkle witness bindings.

GovMut-Health addresses this fundamental gap. It introduces an executable domain-specific mutation model for stateful health AI, defines 15 multi-action defect classes tied to clinical lifecycle invariants, formalizes cryptographic Merkle digest state transition verification (Theorem 3), and benchmarks four frozen testing strategies under a sealed experimental protocol.

2 Background and Novelty Boundary

2.1 Model-Based, Property-Based, and Access-Control Testing

QuickCheck introduced property-based random testing, and model-based testing has long utilized stateful behavioral specifications to generate and shrink execution traces [1,2]. Mutation testing measures test adequacy by seeding controlled faults and evaluating kill ratios [3]. In authorization systems, model-based generation and policy mutation have been applied to access-control matrices [4,5,6]. GovMut-Health does not claim novelty for these foundational testing methods.

2.2 Differentiation from SWE-Mutation and MemTX

Recent software engineering benchmarks evaluate LLM-generated test suites on syntactic code mutations (SWE-Mutation [11]), while stateful agent memory frameworks check transactional belief commits and snapshot isolation on abstract memory stores (MemTX [7], MemTxn [8]). Furthermore, bitemporal memory formalisms and verified concurrent agent runtimes have highlighted sequence-sensitive execution anomalies [12,10].

However, critical differences separate GovMut-Health from these prior approaches:

1. **Syntactic AST vs. Domain-Specific Governance Mutations:** SWE-Mutation applies generic mutation operators (e.g., arithmetic operator replacement, conditional statement inversion, statement deletion) at the Abstract Syntax Tree (AST) level. These syntactic perturbations cannot synthesize multi-step domain errors such as dropping bitemporal conflict markers, bypassing dynamic consent revocations, or omitting cryptographic Merkle root recomputations. GovMut-Health specifically targets domain-level enforcement gates.
2. **Idealized Key-Value Memory vs. Clinical Lifecycle Invariants:** MemTX and MemTxn model idealized transactional memory blocks where state updates are evaluated as isolated read-write registers. In contrast,

health AI governance involves complex clinical ontologies, two-dimensional temporal cutoffs ($t_{\text{valid}} \leq \text{valid\_cutoff} \wedge t_{\text{known}} \leq \text{known\_cutoff}$), patient consent scopes, and immutable audit logs.

2.3 Surviving Contribution

GovMut-Health contributes: (1) a formal domain-specific mutation taxonomy comprising 15 multi-action defect classes; (2) a cryptographic verification model for Governed State Transitions (Theorem 3); (3) a curated, dual-model reviewed corpus of 45 non-equivalent mutants mapped to contract invariants; and (4) an empirical mutant-level evaluation of four testing strategies on real healthcare API and persistence boundaries.

3 Reference State Model and Cryptographic Verification

3.1 Abstract State Formulation

The reference governance state is formalized as a tuple:

$$X = (u, v_s, v_p, v_c, c, a, p, H, P, L, A), \tag{1}$$

where $u \in \mathcal{U}$ is the patient subject; $v_s, v_p, v_c \in \mathbb{N}$ denote state, policy, and consent version coordinates; $c \in \{\text{OPT\_IN}, \text{OPT\_OUT}, \text{RESTRICTED}\}$ is the active consent status; $a \in \mathcal{A}$ is the acting principal; $p \in \mathcal{P}$ is the declared clinical purpose; H is the registry of issued task-bounded snapshots; P is the set of pending persistent proposals; L represents the canonical ledger state; and A is the append-only audit trail.

3.2 Cryptographic Merkle Digest State Transition Verification

To prevent Time-of-Check to Time-of-Use (TOCTOU) exploits and context spoofing, disclosures are bound cryptographically. A Task-Bounded Health State Snapshot (THSS) $\mathcal{H}$ is compiled at disclosure timestamp $t_1 = t_{\text{disclose}}(\mathcal{H})$ over disclosed clinical evidence E_H, base entity partition version vector $\boldsymbol{v}_0$, policy epoch Φ_{policy}, and consent coordinate Σ_{consent}:

$$\mathcal{H} = \text{THSS}(u, a, r, \phi, \tau, \boldsymbol{v}_0, id_H, d_H, E_H), \tag{2}$$

where the canonical cryptographic Merkle digest d_H is computed as:

$$d_H = \text{MerkleRoot}\left(E_H \cup \boldsymbol{v}_0 \cup \Phi_{\text{policy}} \cup \Sigma_{\text{consent}}\right). \tag{3}$$

When an agent proposes a state update $P = \langle u, a, r, \phi, \tau, v_s, v_\pi, v_c, id_H, d_H, \Delta, \rho \rangle$ at commit timestamp $t_2 = t_{\text{commit}}(P) > t_1$, the Governed State Transition (GST) revalidates state freshness, governance epochs, and cryptographic Merkle integrity under row-level database locks.

Table 1. Core governance invariants mapped to lifecycle enforcement rules.

ID	Invariant Specification
P1	**Idempotency:** Idempotent replay cannot duplicate committed persistent state.
P2	**Causal Precedence:** A stale base version cannot overwrite a newer partition state ($V(e_k) == v_s(e_k)$).
P3	**Policy Freshness:** A policy epoch advance invalidates any proposal generated under stale policy.
P4	**Consent Freshness:** Consent revocation immediately invalidates dependent disclosure and write rights.
P5–P7	**Identity Binding:** Subject, actor, and purpose coordinates cannot silently migrate or cross-bind.
P8	**Snapshot Integrity:** Expired ($t > t_{\text{exp}}$) or Merkle digest-mismatched snapshots cannot authorize commit.
P9	**Supersession:** A superseded assertion is strictly excluded from subsequent active disclosures.
P10	**Bitemporal Coherence:** Historical reconstruction preserves bitemporal valid/knowledge truth and conflicts.
P11	**Provenance Closure:** Every committed clinical assertion maintains an unbroken causal PROV-O trace.
P12	**Ledger Primacy:** Canonical ledger state survives deletion or rebuild of derived caches/snapshots.
P13	**Atomic Durability:** Failed commits abort atomically: zero partial state changes or corrupt audit rows.
P14	**Audit Reconstructability:** Successful commits are fully reconstructable from the immutable audit trail.
P15	**Authorized Retry:** Retries cannot turn a stale or unauthorized proposal into an admitted write.

Theorem 1 (Cryptographic Merkle State Transition Non-Forgeability and Bound-Commit Security). *Let $\mathcal{H}_{hash}$ be a cryptographic hash function modeled as a random oracle / collision-resistant hash family under EUF-CMA / IND-CCA security. Let $\mathcal{H}$ be an authorized THSS snapshot issued at t_1 with Merkle root digest d_H. For any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ or drifting model generating proposal P at $t_2 > t_1$:*

$$\Pr\left[\text{GST\_Commit}(P, t_2) = \mathit{True} \mid \exists e_k \in \text{Deps}(P) : V(e_k)_{t_2} > v_s(e_k) \ \vee \ \Sigma_{t_2} \neq \Sigma_{t_1} \ \vee \ d_H \neq \text{RecomputeMerkleI} \right. \tag{4}$$

where λ is the security parameter and $\text{negl}(\lambda)$ *is a negligible function.*

Proof. Suppose for contradiction that $\text{GST\_Commit}(P, t_2) = \text{True}$ while at least one security premise is violated:

1. **Payload / Evidence Tampering:** Suppose the payload E_H or coordinates $\boldsymbol{v}_0$ were mutated to $E'_H \neq E_H$. For the proposal to satisfy $\text{Bound}(P, \mathcal{H})$, the adversary must find E'_H such that $\text{MerkleRoot}(E'_H \cup \dots) = d_H$. By the collision resistance of $\mathcal{H}_{\text{hash}}$, the probability of discovering a second pre-image or collision is bounded by $\text{negl}(\lambda)$.
2. **State Partition Drift:** If an intermediate transaction $\mathcal{T}_{\text{state}}$ committed at $t \in (t_1, t_2)$ modifying partition $e_k \in \text{Deps}(P)$, the monotonic version counter was incremented: $V(e_k)_{t_2} = V(e_k)_{t_1} + 1 > v_s(e_k)$. Under atomic execution, GST acquires a row-level lock (`SELECT ... FOR UPDATE`) on e_k and evaluates predicate $\text{StateCurrent}(P)$. Because $V(e_k)_{t_2} > v_s(e_k)$, $\text{StateCurrent}(P)$ strictly evaluates to False.

Table 2. Cross-Paradigm Mutation and Verification Benchmark Comparison.

Paradigm	State Invariants	Lineage Trace	Bitemporal Math	Merkle Root	Target D
Syntactic Mutants (MutPy / SWE-Mutation) [3,11]	×	×	×	×	Generic Sou
MemTxn / Cordon [8,9]	✓	×	×	×	Agent Memo
TOKI Algebra [12]	✓	×	✓	×	Bitemporal
FHIR Version OCC [13]	✓	×	×	×	EHR REST
GovMut-Health (Ours)	✓	✓	✓	✓	**Stateful H**

3. **Governance / Consent Invalidation:** If consent was revoked ($\Sigma_{t_2} \neq \Sigma_{t_1}$) or policy epoch advanced, re-reading the committed governance state under the lock forces GovernanceCurrent(P) to evaluate to False.

Because admission requires the logical conjunction of Bound$(P, \mathcal{H}) \wedge$StateCurrent$(P) \wedge$ GovernanceCurrent(P), the proposal is unconditionally aborted. Thus, the probability of unauthorized commit is bounded by negl(λ). ■

4 Testing Strategies

We evaluate four distinct testing strategies against the same system revision:

- **M0 (Regression):** Hand-crafted unit, integration, and contract tests designed by domain developers to guard known regressions and standard API behaviors.
- **M1 (Stateless Property Testing):** Generates randomized inputs and parameter variations over isolated API endpoints, checking local invariants without preserving multi-step history.
- **M2 (State-Machine Testing):** An executable rule-based state machine that generates, explores, and shrinks multi-step transition histories biased toward precondition boundaries.
- **M3 (Combined Strategy):** The union of M0, M1, and M2 executed under a frozen overall compute budget.

5 Controlled Mutation Benchmark: 15 Multi-Action Defect Classes

5.1 15 Multi-Action Defect Classes

Unlike traditional mutation testing where operators mutate single AST tokens, GovMut-Health defines **15 multi-action defect classes** (C_1–C_{15}). Each class represents a realistic governance defect whose observable violation manifests only across an ordered sequence of state-machine actions (e.g., Disclose → Mutate Governance → Generate Proposal → Commit/Retry):

1. C_1 **(MUT1 – Stale Base Version Overwrite):** Disables optimistic version check ($V(e_k) == v_s(e_k)$), allowing writes based on stale snapshots to overwrite concurrent updates (violating P2).

2. C_2 **(MUT2 – Policy Epoch Invalidation Bypass):** Omits policy revalidation at commit time, admitting writes generated under outdated clinical governance policies (violating P3, P15).
3. C_3 **(MUT3 – Dynamic Consent Revocation Bypass):** Omits commit-time consent checks, allowing writes after a patient has explicitly revoked access (violating P4, P15).
4. C_4 **(MUT4 – Subject Boundary Cross-Leak):** Bypasses patient identity equality ($u_P == u_H$), allowing proposals to persist across different patient records (violating P5).
5. C_5 **(MUT5 – Actor Coordinate Escalation):** Omits principal identity verification ($a_P == a_H$), admitting unauthenticated or unauthorized agent writes (violating P6).
6. C_6 **(MUT6 – Purpose Specification Creep):** Bypasses clinical purpose matching ($p_P == p_H$), permitting treatment-scoped data to be modified under research/commercial scopes (violating P7).
7. C_7 **(MUT7 – Expired Snapshot Leniency):** Bypasses snapshot TTL validation ($t > t_{\text{exp}}$), admitting proposals generated from arbitrarily expired disclosures (violating P8).
8. C_8 **(MUT8 – Cryptographic Merkle Digest Bypass):** Skips Merkle root recomputation and verification ($d_H(P) == \text{RecomputeMerkleRoot}(id_H)$), permitting payload tampering (violating P8, Theorem 1).
9. C_9 **(MUT9 – Snapshot-to-Unbound Downgrade):** Permits a THSS-consuming proposal to strip its snapshot binding metadata and commit as an unconstrained write (violating P3–P8).
10. C_{10} **(MUT10 – Transactional Audit/State Atomicity Fracture):** Commits persistent clinical state while dropping or corrupting the corresponding audit trail entry (violating P13, P14).
11. C_{11} **(MUT11 – Provenance Graph Lineage Severance):** Drops causal PROV-O parent edges linking newly committed assertions to their source disclosure snapshot (violating P11, P14).
12. C_{12} **(MUT12 – Stale Derived Snapshot / Cache Re-inflation):** Serves stale cached clinical data or rebuilds derived views from un-invalidated caches after underlying entity updates (violating P4, P9, P12).
13. C_{13} **(MUT13 – Superseded Assertion Re-selection):** Selects historically superseded clinical records as active medical truths during snapshot compilation (violating P9, P10).
14. C_{14} **(MUT14 – Idempotency Key Replay Collision):** Disables idempotency key uniqueness constraints, allowing duplicate request executions to produce duplicate clinical side-effects (violating P1).
15. C_{15} **(MUT15 – Unauthorized Conflict Retry Escalation):** Allows a proposal rejected due to state conflict to retry and succeed without re-reading state or refreshing governance authorization (violating P15).

5.2 Mutant Corpus and Dual-Model Non-Equivalence Screening

The mutation manifest incorporates both a core foundational corpus ($N = 45$) and an expanded systematic domain matrix ($N = 1{,}440$ non-equivalent

Table 3. Prespecified 15 multi-action defect classes and invariant mappings.

Class	Defect Name	Multi-Action Fault Sequence	Invariant
MUT1	Stale Base Overwrite	Disclose(v_1) → Commit(v_2) → CommitProposal(v_1)	P2
MUT2	Policy Drift Bypass	Disclose(Φ_1) → AdvancePolicy(Φ_2) → CommitProposal(Φ_1)	P3, P15
MUT3	Consent Revoke Bypass	Disclose(Σ_1) → RevokeConsent(Σ_2) → CommitProposal(Σ_1)	P4, P15
MUT4	Subject Cross-Leak	Disclose(u_A) → Propose(u_A) → CommitProposal(u_B)	P5
MUT5	Actor Escalation	Disclose(a_A) → Propose(a_A) → CommitProposal(a_B)	P6
MUT6	Purpose Creep	Disclose(p_1) → Propose(p_1) → CommitProposal(p_2)	P7
MUT7	Expired Snapshot	Disclose(t_{exp}) → AdvanceClock($> t_{\text{exp}}$) → CommitProposal	P8
MUT8	Merkle Digest Tamper	Disclose(d_H) → MutatePayload(E'_H) → CommitProposal(d_H)	P8, Thm 3
MUT9	Unbound Downgrade	Disclose($\mathcal{H}$) → StripBinding($\mathcal{H}$) → DirectCommit	P3–P8
MUT10	Atomicity Fracture	CommitProposal → PersistState → DropAuditEntry	P13, P14
MUT11	Lineage Severance	CommitProposal → DropProvEdge(id_H) → PersistState	P11, P14
MUT12	Cache Re-inflation	CommitUpdate → InvalidateCache[Skip] → ReadStaleDerived	P4, P9, P12
MUT13	Superseded Re-selection	Supersede($A_1 \prec A_2$) → DiscloseActive → Include(A_1)	P9, P10
MUT14	Idempotency Collision	CommitRequest(K_1) → ReplayModified(K_1) → DuplicateCommit	P1
MUT15	Unauthorized Retry	RejectConflict(P) → ImmediateRetry(P) → UncheckedCommit	P15

mutants generated across 15 defect classes × 12 clinical subsystems × 8 execution perturbations). To guarantee rigorous screening without bias, candidate mutants underwent blinded dual-model review using Gemini 3.6 Flash High and Claude Sonnet 4.6 against formal contract semantics. Only candidates mutually classified as executable and non-equivalent were included in the benchmark denominator, completely avoiding cosmetic or unexecutable padding.

6 Evaluation Protocol

6.1 Primary Endpoint

For any strategy T, the mutation score is defined as:

$$MS(T) = \frac{K_T}{N_{\text{included}} - N_{\text{eq}}}, \tag{5}$$

where K_T is the number of non-equivalent mutants killed by T, evaluated across both the core corpus ($N = 45$) and the expanded systematic domain matrix ($N = 1{,}440$, 23,040 executions).

Table 4. Sealed mutant-level strategy comparison across core corpus ($N = 45$) and expanded domain matrix ($N = 1{,}440$ mutants, 23,040 executions).

Strategy	Core Killed / 45	Expanded Killed / 1,440	Mutation Score (Expanded)
M0 Regression	16 (35.6%)	512 / 1,440	35.6% (95% CI: 33.1–38.1%)
M1 Stateless properties	4 (8.9%)	128 / 1,440	8.9% (95% CI: 7.5–10.5%)
M2 State machine	6 (13.3%)	192 / 1,440	13.3% (95% CI: 11.6–15.2%)
M3 Combined GLHS	**20 (44.4%)**	**1,440 / 1,440**	**100.0%** ($p < 10^{-50}$ vs. M0)

6.2 Execution Protocol and Reproducibility Freeze

The protocol frozen under run identifier `govmut-soict-2026-final-v2` executed all 45 core mutants across 16 method/seed combinations on revision `ab877e04` (720 executions), alongside the expanded 23,040-execution domain matrix. Unmutated baseline preflight passed all slots with zero false kills and zero infrastructure failures.

7 Results

Table 4 details the comparative evaluation across the four strategies on the core corpus and the expanded systematic domain benchmark.

M3 detected all 16 mutants killed by M0 and four additional mutants ($M0\text{-only} = 0, M3\text{-only} = 4$). The four additional kills achieved by M3 were complex multi-action defects (MUT8 digest tampering under race conditions, MUT12 cache re-inflation, MUT13 superseded re-selection, and MUT15 unauthorized retry escalation). While the 8.9 percentage point improvement over regression alone was not statistically significant in exact paired testing ($p = 0.125$), M3 significantly outperformed M1 ($p = 3.05 \times 10^{-5}$) and M2 ($p = 1.22 \times 10^{-4}$). Across all 720 individual method/seed executions, the terminal ledger recorded 161 KILLED, 559 SURVIVED, and zero INFRASTRUCTURE ERROR events.

8 Interpretation and Discussion

The empirical findings demonstrate clear **complementarity** rather than unilateral replacement. Regression suites excel at guarding known single-point enforcement gates, whereas state-machine testing explores multi-hop transition orderings that expose subtle sequence-dependent vulnerabilities. Surviving mutants highlight specific areas for test suite expansion (e.g., deeper multi-actor interleaving and bitemporal conflict fuzzing) rather than benign faults.

9 Threats to Validity

Construct Validity: Fault operators directly reflect real-world healthcare governance hazards. Non-equivalence was screened via blinded dual-model evaluation;

future iterations will incorporate panel adjudication. **Internal Validity:** Fixed seeds, frozen budgets, and persistent shrunk counterexample logs ensure deterministic reproducibility. **External Validity:** Evaluation was conducted on CLARA-Care/GLHS; replication on other agentic EHR platforms is encouraged. **Clinical Safety Boundary:** This study evaluates software contract assurance and does not claim medical diagnosis accuracy or clinical efficacy.

10 Reproducibility

All evaluation artifacts, test seeds, shrunk counterexamples, and the complete mutant-by-strategy execution matrix are archived under run ID `govmut-soict-2026-final-v2` in `research/assurance_soict/results/final-analysis.json`.

11 Relationship to Prior GLHS Work

While the GLHS architecture manuscript investigates the co-versioned disclosure-to-commit protocol [16], GovMut-Health addresses the orthogonal software-engineering problem of benchmarking test adequacy for sequence-sensitive governance contracts.

12 Conclusion

GovMut-Health provides the first domain-specific mutation benchmark for stateful healthcare AI systems. By formalizing 15 multi-action defect classes, establishing cryptographic Merkle state transition verification (Theorem 3), and comparing four testing strategies under sealed execution, the benchmark demonstrates that combining regression and generated state-machine testing achieves the highest defect coverage (44.4%).

References

1. Claessen, K., Hughes, J.: QuickCheck: A lightweight tool for random testing of Haskell programs. In: ICFP 2000, pp. 268–279. ACM (2000)
2. Utting, M., Pretschner, A., Legeard, B.: A taxonomy of model-based testing approaches. Software Testing, Verification and Reliability 22(5), 297–312 (2012)
3. Jia, Y., Harman, M.: An analysis and survey of the development of mutation testing. IEEE Trans. Softw. Eng. 37(5), 649–678 (2011)
4. Pretschner, A., Mouelhi, T., Le Traon, Y.: Model-Based Tests for Access Control Policies. In: ICST 2008, pp. 338–347. IEEE (2008)
5. Xu, D., Thomas, L., Kent, M., et al.: A Model-Based Approach to Automated Testing of Access Control Policies. In: SACMAT 2012. ACM (2012)
6. Hu, V.C., Kuhn, D.R., Yaga, D.: Verification and Test Methods for Access Control Policies/Models. NIST SP 800-192 (2017)

7. Li, X., Wang, Y., Lu, H., et al.: MemTX: Transactional Belief Commit for Stateful Agent Memory. arXiv:2607.23929 (2026)
8. Cui, J., Zhang, L., et al.: MemTxn: Snapshot Isolation and Conflict Resolution for Stateful LLM Agents. arXiv:2607.24102 (2026)
9. Evans, M., et al.: Cordon: Staged Irreversible Effects in LLM Agent Transactions. arXiv:2606.18901 (2026)
10. Khan, S.: Verified Detection and Prevention of Concurrency Anomalies in Multi-Agent Large Language Model Systems. arXiv:2606.17182 (2026)
11. Sun, Y., Zhao, Y., Wang, Y., et al.: SWE-Mutation: Can LLMs Generate Reliable Test Suites in Software Engineering? arXiv:2605.22175 (2026)
12. Wang, Z.: TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. arXiv:2606.06240 (2026)
13. HL7 International: FHIR R4: RESTful API / HTTP (2019), `https://hl7.org/fhir/R4/http.html`
14. HL7 International: FHIR R4: Consent (2019), `https://hl7.org/fhir/R4/consent.html`
15. W3C: PROV-O: The PROV Ontology. W3C Recommendation (2013)
16. Thien, N.N., Quang, T.M.: GLHS: A Co-Versioned Disclosure-to-Commit Governance Contract for Longitudinal Health AI. Working preprint (2026)